Lab 21: Review and test generation fight over one Claude call

Topic 2: Tool Design & MCP Integration · Task 2.3: Distribute tools appropriately across agents and configure tool choice

Runs on Simulator 5: Claude Code for Continuous Integration · Run in class

Source: SkillCertPro practice exam Part 1, Q15 (unofficial third-party question)

Commit to an answer first

The system uses one Claude call for pull request review, test generation, and PR metadata updates. Logs show the review phase sometimes invokes test-writing tools for non-testable style comments, then returns prose instead of the structured finding object your pipeline expects. What change best improves tool-selection reliability while preserving automated test generation elsewhere in the pipeline?

A. Keep one shared CI step, expose all tools with sharper descriptions, and set tool_choice to "any" for review output.

B. Keep one shared CI step, namespace tool names by category, and leave tool_choice on "auto" for model flexibility.

C. Expose all CI tools to both steps, set tool_choice to "none" during review, and parse JSON from assistant text.

D. Separate review and test-generation steps, expose only role-specific tools to each step, and force the review step's reporting tool with tool_choice .

Simulate the solution

```
# In the browser: open Simulator 5, then Lab 21
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 21

# In a terminal, from simulators/simulator-5-ci-cd/
uv run casekit.py --case UC-15
```

What you should see

Review gets read-only tools and the finding schema. Tests get their own. No prose comes back.

The review step gets read-only tools and the finding schema; the test step gets its own schema. Each returns structured output, never prose. In the Messages API the same guarantee is tool_choice {type: tool, name: report_findings}.

Open `simulators/simulator-5-ci-cd/evidence/` and find it yourself.

The answer

D. Separate steps, role-specific tools, a forced reporting tool for review.

Scope tool access to the role performing each step. A review step that only produces findings should not hold test-writing or PR-label tools, because every extra tool raises the chance of an irrelevant call. Descriptions are the main steering mechanism, but they are no substitute for removing unrelated tools. tool_choice: {"type": "tool", "name": …} forces the named reporting tool, so the pipeline gets structured findings, while a separate test-generation step keeps its file-writing tools.

Why the others fail

A. "any" guarantees a tool call, which could still be a test-writing tool.

B. "auto" still allows prose, and namespacing does not remove the unrelated tools.

C. "none" forbids tools, so findings come back as prose to parse. That gives up the structured output.

Check yourself

- the review step returned a schema-valid finding object, not prose
- the review step reported the loop bug and nothing about tests
- the test step returned its own contract with the duplicate skipped

Now change one thing

Fix the loop in repo/stock/totals.py and the caller in repo/stock/jobs.py, rerun, and watch the blocking count fall.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.